Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Artificial Intelligence ENCS3340

Project #1

Optimization Strategies for Local Package Delivery Operations

Partners:

Malak Milhem - 1220031

Maysam Habbash - 1220075

Section: 3

Instructor: Samah Alaydi

May 3, 2025

Introduction

In this project we implemented two artificial intelligence algorithms to find solutions to local package delivery operations. The objective was to minimize the total distance traveled by delivery vehicles while ensuring no vehicle exceeds its capacity and higher-priority packages are delivered first whenever possible. GA evolves a population of potential solutions using selection, crossover, and mutation operators, while SA explores the solution space through probabilistic acceptance of neighbor solutions, guided by a temperature-based cooling schedule. Both algorithms proved their ability to handle constraints, adapt to parameter tuning, and generate efficient, practical delivery assignments.

Genetic Algorithm

The approach we followed for bringing the genetic algorithm from theory to practice in our package to vehicle assignment optimization problem consists of multiple organized steps:

1. **Generating Individuals**

An individual represents a possible solution to the problem we are presenting. We formulate a solution as a valid assignment of packages to vehicles, where the condition of no load on a vehicle exceeds its capacity is preserved, and the packages in each vehicle are initially sorted by priority for delivery.

We used a generate_individual function that takes packages and vehicles as arguments and returns a dictionary of vehicle IDs pointing each to a corresponding list of packages. A number of individuals is generated to form an initial population of fixed size.

2. **Evaluating Individuals**

Each individual (Solution) is evaluated based on the total cost taken by all vehicles. If a violation to the priority constraint occurs, a fixed cost penalty is added to the total cost to avoid critical difference in costs. The evaluate_individual function we used takes individual as an argument and returns a tuple of individual and a total cost (individual, cost).

3. **Selection Operator**

We applied a section operator to our population to favor exploration and exploitation of the domain. To achieve this, we implemented a system of tournaments, in which a subset of the population, of tournament size, is randomly selected, then, a winner that is most fitted, meaning it has lower total cost given the evaluation of it, is chosen and returned. This enables the best solutions to be more dominant in the next generations, increasing the probability of finding a more optimal solution.

4. **Crossover Operator**

To further favor exploration of the domain, we applied the crossover operator to individuals in the population. We had this implemented by taking half of the vehicle routes from one parent individual and the other half from another parent and merging them into a child individual, while making sure no package gets assigned more than one, and no packages are left unassigned.

5. **Mutation Operator**

As for mutations, we made use of this operator to violate the constraint of having higher priority packages delivered first so we increase the chance of getting more optimal solutions. This implementation considered reordering of packages in a vehicle when there is a chance.

6. **Parameters**

We tried different population and tournament sizes. A bigger population gave more variety in solutions but took more running time. Smaller tournaments helped explore new options, while bigger ones focused on picking the best. We found a good balance by testing different settings.

Simulated Annealing Algorithm

In our project, SA was used to find an optimized assignment of packages to vehicles, aiming to minimize total distance traveled while respecting constraints like vehicle capacity and delivery priority.

1. Generating Initial State

The algorithm begins by generating a valid initial state. a function creates a state which represents vehicles with the packages it holds in the order they will be delivered in. respecting vehicle capacities and prioritizing high-priority packages when possible. This initial state is constructed randomly but is validated to ensure feasibility.

2. Evaluation Function

An evaluation function is implemented to individuals as mentioned before, the result of the evaluation determines the possibility of committing to the individual as a solution. If a state violates constraints such as overloading a vehicle or misprioritizating packages, a penalty is added to its cost to discourage the algorithm from selecting it. Lower-cost states are considered better solutions.

3. Neighbor Generation

neighbor states are generated by making small changes to the current state to explore the solution space. This could involve swapping packages between vehicles, changing delivery orders, or moving a package to a different vehicle, again while ensuring constraints are not violated.

4. Acceptance Criteria

A neighbor state is accepted based on two factors, If it has a lower cost than the current state, it is always accepted. And If it has a higher cost, it may still be accepted with a certain probability, which decreases over time as the system "cools." This probability is controlled by a temperature parameter that decreases according to a cooling schedule. This mechanism helps the algorithm avoid getting stuck in local optima.

5. Cooling Schedule

We used an exponential cooling schedule ($e^{\frac{\Delta E}{T}}$) where the temperature is reduced after each iteration. The algorithm terminates either when a minimum temperature is reached (Tmin =1) or after a fixed number of iterations(max iterations = 100).

6. **Parameters**

Simulated annealing parameters are: initial Temperature, Cooling Rate, stopping Temperature and Number of Iterations per Temperature. Each parameter has an effect on the algorithm's solution's quality. For example, for the number of iterations Increasing iterations helped better explore each temperature level, but the gain in solution quality diminished beyond 100 iterations. And a slower cooling rate (0.99) prolonged the search and produced better solutions but increased runtime. Thus a cooling rate of 0.95 was a good compromise for us.

Test Case #1

- ## Input Files
    - ### Packages

Priority weight x-coordinate y-coordinate

4 5 20 20

0 80 45 5

2 3 30 5

1 1 50 50

3 7 0 10

3 15.6 12 8

2 8.3 23 45

4 19.5 10 10

0 80 45 4

2 30 30 5

1 10 50 50

    - ### Vehicles

10 vehicles each of capacity 100

- ## Parameters

Initial temperature = 1000

Stopping temperature = 1

Cooling rate = 0.95

Iterations = 100

Population size = 75

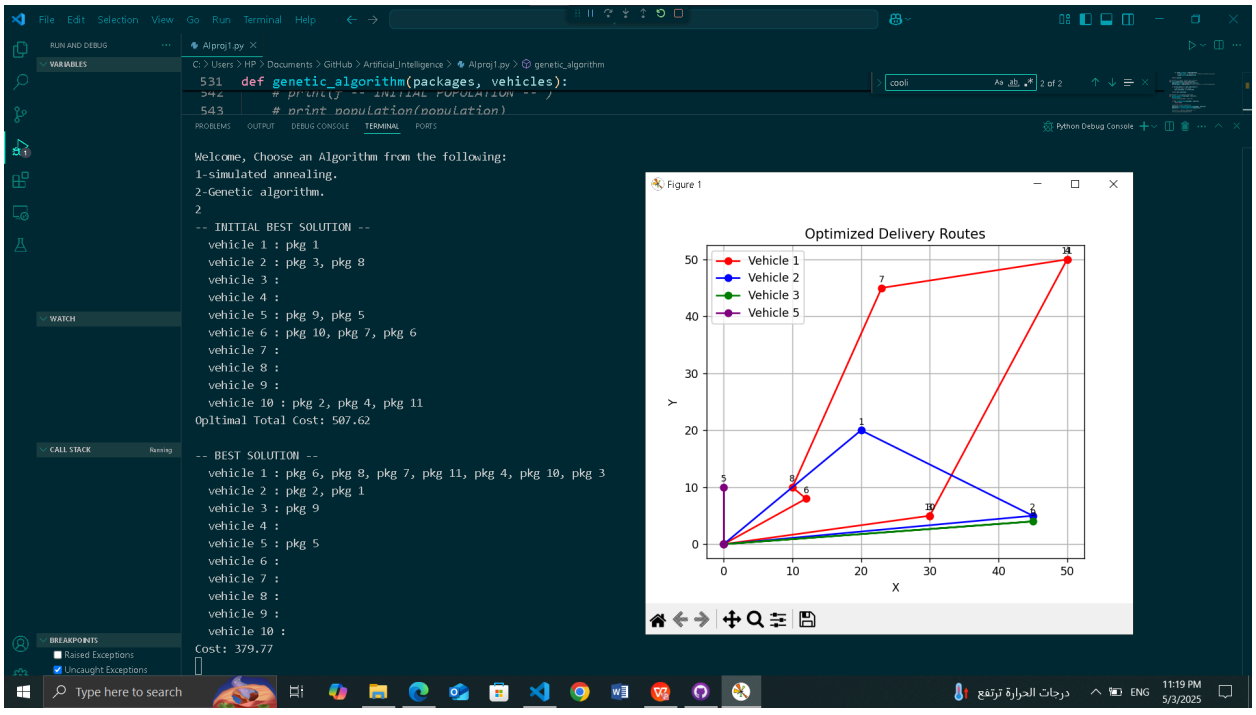
Tournament ratio = 0.2

Number of generations = 500

Mutation rate = 75

- Result of Simulated Annealing Algorithm

- Result of Genetic Algorithm



Test Case #2

- Input Files
    - Packages

Priority weight x-coordinate y-coordinate

4 5 20 20

0 115 45 5

2 3 30 5

    - vehicles

10 vehicles each of capacity 100

- Result (Terminal View)